Deepfake Image Detection Using EfficientNet-B0

Deepfake ML Hackathon – Synergy '26 | Team: Padmini and Meghana | Google Colab, NVIDIA Tesla T4 | PyTorch 2.11.0, Torchvision 0.26.0

Abstract

We built a binary deepfake image classifier (real = 0, fake = 1) by fine-tuning an ImageNet-pretrained EfficientNet-B0 on the hackathon training data. Images are resized to 224 × 224 and normalized with ImageNet statistics. We train with AdamW and binary cross-entropy with logits, use an 80/20 stratified split, and keep the checkpoint with the highest validation AUC-ROC. Our **best validation AUC-ROC is 0.8925 (epoch 8)**. Inference on a Tesla T4 takes about **15 seconds for 3,000 images**, including model loading. Our inference script matches the official template's arguments and output columns.

1. Dataset and Validation Strategy

We used 7,000 labelled images (3,522 fake, 3,478 real), split 80/20 with stratification (random_state 42) into 5,600 training and 1,400 validation images. Every image is converted to RGB, resized to 224 × 224, turned into a tensor and normalized with ImageNet means [0.485, 0.456, 0.406] and standard deviations [0.229, 0.224, 0.225]; the same steps are used in training, validation and inference. **Validation strategy:** the 20% hold-out set is used only to score each epoch with AUC-ROC (computed from sigmoid probabilities) and to select the best checkpoint; it is never used for training.

2. Data Augmentation

Applied to training images only: random horizontal flip; ColorJitter with brightness and contrast 0.1; Gaussian blur with probability 0.2 and radius randomly chosen between 0.5 and 1.5. Validation data is not augmented.

3. Model Architecture and Training

We use Torchvision EfficientNet-B0 with ImageNet1K_V1 weights and replace the classifier with Dropout (0.3) followed by a linear layer from 1,280 features to one logit. The sigmoid of the logit gives the fake-class probability p; the label is 1 when p ≥ 0.5, otherwise 0. The confidence field stores p and is not a calibrated certainty. Training used CUDA mixed precision (autocast and gradient scaling) on a single T4 GPU with all seeds set to 42.

Parameter	Value	Parameter	Value
Backbone	EfficientNet-B0 (ImageNet1K_V1)	Learning rate	0.0003
Input size	224 × 224	Optimizer / weight decay	AdamW / 0.0001
Epochs / batch size	10 / 32	Loss	BCEWithLogitsLoss
Dropout	0.3	Checkpoint rule	Best validation AUC-ROC

4. Results

Training loss fell from 0.5270 (epoch 1) to 0.0811 (epoch 10). Validation AUC-ROC stayed between 0.8738 and 0.8925 and peaked at **0.8925 at epoch 8**; it was 0.8832 at epoch 1 and 0.8877 at epoch 10. This value is measured on our own validation split and is not the hidden-test score. AUC-ROC measures ranking quality across thresholds and is not accuracy. Because training loss keeps falling while validation AUC stays flat, this suggests possible overfitting.

Epoch	1	2	3	4	5	6	7	8	9	10
Train loss	0.5270	0.3795	0.2911	0.2196	0.1774	0.1480	0.1112	0.0958	0.0838	0.0811
Val AUC-ROC	0.8832	0.8889	0.8861	0.8738	0.8803	0.8868	0.8876	**0.8925**	0.8827	0.8877

5. Inference, Efficiency and Robustness

The inference script takes `--test_dir`, `--weights` and `--output`. It rebuilds the model head, loads the saved weights, runs in evaluation mode with batch size 64, processes images sorted by filename, and writes a CSV with columns `image_id, label, confidence`. Our `predictions.csv` has 3,000 rows with no missing values or duplicate IDs, and labels are consistent with the 0.5 threshold. On a Tesla T4, 3,000 images took about 15 seconds including model loading, well under the five-minute limit. Gaussian blur during training exposes the model to some degradation, but we did not measure robustness to JPEG compression, noise or resizing, so we make no claim about it.

6. Limitations

(i) Validation AUC peaks at 0.8925 while training loss keeps decreasing, suggesting possible overfitting; (ii) accuracy, precision, recall and F1-score were not calculated; (iii) no numerical robustness results for JPEG, noise or resizing; (iv) peak GPU memory was not measured.